

Dónde vive cada cosa

La misma guía de diagnóstico, con el archivo, la función y la tabla concreta al lado de cada explicación. Para señalar un fallo con precisión, o para dárselo a quien vaya a tocar el código.

Verificado sobre `main`, esquema 50. La versión sin jerga es `Diagnostico.pdf`.

1. Levantar el proyecto desde cero

Lo mínimo para tener Noesis corriendo en una máquina limpia.

```
git clone https://github.com/noesisstudio/noesis
cd noesis
py -m venv .venv
.\.venv\Scripts\Activate.ps1      # Windows
pip install -e .
noesis-web                        # http://127.0.0.1:8000
```

Extras opcionales, según lo que vayas a tocar:

```
pip install -e ".[test]"          # pytest y hypothesis
pip install -e ".[analysis]"      # regenerar el modelo económico
pip install -e ".[security]"      # ruff, bandit, detect-secrets
pip install -e ".[audio]"         # transcripción local con faster-whisper
```

Sin `DATABASE_URL` usa SQLite en un archivo local, y el esquema se aplica solo al arrancar. Con ella, PostgreSQL. Para trabajar sin tocar tus datos, apunta `NOESIS_DB_PATH` a otro archivo.

COMANDO	PARA QUÉ
<code>noesis-web</code>	Levanta el servidor
<code>noesis</code>	El asistente por consola, sin navegador
<code>noesis-doctor --strict</code>	Diagnóstico de preparación: qué falta para producción
<code>noesis-restore-check</code>	Comprueba que una copia se puede restaurar
<code>python -m pytest -q</code>	La suite: 531 pruebas
<code>python scripts/check_project_truth.py</code>	Verifica que la documentación no miente sobre el código

2. El recorrido de una petición

Cuatro puertas, en este orden. Saber en cuál cae el rechazo es el diagnóstico.

PUERTA	DÓNDE VIVE	QUÉ HACE SI RECHAZA
Identidad	<code>web/auth.py</code> → <code>current_user</code>	Limpia la sesión y redirige a <code>/login</code> . Comprueba <code>session_version</code> y que la cuenta esté activa
Aislamiento	<code>web/deps.py</code> → <code>auth_guard</code>	Compara el negocio del usuario con el de la URL. Redirige a su panel, o 403 en <code>/api/</code>
Permisos del plan	<code>adapters/billing.py</code> → <code>entitlements_for</code>	Redirige a suscripción con <code>status=upgrade</code>
Estado de la suscripción	<code>db.py</code> → <code>subscription_allows_acc</code> <code>ess</code>	Deja leer, bloquea escribir. Es el modo consulta

El punto que no se puede tocar a la ligera

`auth_guard` es el **único** sitio que autoriza `/b/` y `/api/` en todo Noesis. Es la garantía de aislamiento entre negocios: un cambio ahí afecta a todos los clientes a la vez.

Antes hay tres middlewares más, en este orden inverso al de declaración:

`security_headers` , `canonical_public_host` y `request_observability` . Un 403 raro con origen correcto suele venir de la comprobación CSRF por cabeceras, en `csrf_response` .

3. El mapa del código

Qué hay en cada sitio, para no buscar a ciegas.

RUTA	QUÉ CONTIENE
<code>db.py</code>	Acceso único a datos. Toda consulta filtra por <code>business_id</code>
<code>migrations.py</code>	Esquema versionado. Hoy, 50. Sube y baja
<code>nlu.py</code>	Entiende las frases: «factura a Juan 95 euros»
<code>internal_brain.py</code>	Redacta sin IA externa: cobros, presupuestos, citas
<code>tools.py</code>	Las acciones que el asistente puede ejecutar
<code>readiness.py</code>	El <code>noesis-doctor</code>
<code>web/deps.py</code>	Guardián, plantillas, filtros como <code>date_es</code>
<code>web/whatsapp.py</code>	Entrada, salida, firma, plantillas y cola
<code>web/scheduler.py</code>	Los trabajos de fondo
<code>web/routers/</code>	15 routers por dominio: cuentas, facturas, documentos, equipo, gestión, admin...
<code>adapters/</code>	Lo externo y reemplazable: <code>ai</code> , <code>billing</code> , <code>email</code> , <code>extraction</code> , <code>invoicing</code> , <code>transcription</code>

Por qué importa la carpeta `adapters/` : todo lo que depende de un tercero está detrás de uno. Si algo externo falla, el fallo está ahí y no repartido por el producto. Es el primer sitio donde mirar cuando el síntoma involucra a un proveedor.

4. Las colas

Nada se envía en caliente. Todo pasa por una cola durable, y ahí es donde se ve qué falló.

COLA	TABLA	ESTADOS
Correo	email_outbox	queued → processing → sent / retrying / failed
WhatsApp	whatsapp_outbox	Igual, más delivered y read desde Meta
Veri*Factu	Cola propia con reintentos	Aceptada, rechazada, con errores

Reintento con espera creciente de 30 s a 1 h, seis intentos. La idempotency_key impide que el mismo aviso salga dos veces. En WhatsApp la cola guarda además connection_id , así que un reintento sale por el mismo número que recibió el mensaje.

Para diagnosticar: db.admin_support_delivery_failures(business_id) devuelve canal, estado, intentos y el error del proveedor, sin exponer destinatario, asunto ni cuerpo. Es lo que pinta «Entregas atascadas» en la ficha de la cuenta.

5. Los trabajos de fondo

Lo que ocurre sin que nadie pulse nada. Si un aviso no llega, empieza por comprobar que su hora pasó.

TRABAJO	CUÁNDO
Resumen diario	Cada día a las 8:00
Recordatorios de cobro	Cada día a las 9:00
Propuestas de cobro	Cada día a las 10:00
Cierre del día	Por la tarde
Resumen semanal	Domingos a las 18:00
Informe al fundador	Lunes a las 9:00
Aviso fiscal	Al cerrar cada trimestre

Cada ejecución se reserva con `claim_scheduled_run`, así que dos servidores no mandan el mismo aviso dos veces. Y solo alcanzan a negocios con WhatsApp conectado y suscripción que permita acceso.

6. De síntoma a archivo

La tabla que convierte una queja en un sitio concreto donde mirar.

SÍNTOMA	CAUSA HABITUAL	DÓNDE
No deja crear	Modo consulta	<code>db.subscription_allows_access</code>
Redirige al login	Sesión inválida o cuenta suspendida	<code>auth.current_user</code> , <code>users.is_active</code>
Redirige a su panel	Pidió otro negocio	<code>auth_guard</code> — es correcto, no un fallo
403 en una API	Aislamiento, o CSRF por cabeceras	<code>auth_guard</code> , <code>csrf_response</code>
Falta una sección	Plan sin ese permiso	<code>billing.PLAN_ENTITLEMENTS</code>
Correo sin llegar	Sin <code>BREVO_API_KEY</code> , o rebote	<code>adapters/email.py</code> , <code>email_outbox.last_error</code>
Voz sin transcribir	<code>get_transcriber()</code> devuelve <code>None</code>	<code>adapters/transcription.py</code>
WhatsApp entra pero no sale	Token sin el activo, o sin método de pago	<code>whatsapp_outbox.last_error</code>
Plantilla rechazada	Nº de variables distinto al aprobado	<code>config.WHATSAPP_TEMPLATE_*</code> , <code>tools.py</code>
Mensaje ignorado sin más	Receptor desconocido o WABA discordante	<code>whatsapp._handle_inbound</code> — deliberado
Pagó y sigue bloqueado	Webhook de Stripe perdido o desordenado	<code>reconcile_stripe_subscription</code>
Fecha rara como <code>2026-07-20T00:00:00</code>	Falta el filtro <code>date_es</code>	La plantilla que la pinta

7. Comprobaciones rápidas

```
curl https://bynoesis.com/health # vivo + release desplegado
curl https://bynoesis.com/ready  # + esquema aplicado
```

Si `/ready` devuelve un esquema distinto al de `project-state.json`, hay un despliegue a medias: ese es el fallo y no lo que te esté contando el cliente.

```
python -m pytest tests/ -k "whatsapp" # acotar por área
python -m ruff check src/              # errores reales, no estilo
```

En Windows fallan tres o cuatro pruebas con `PermissionError [WinError 32]` al borrar el temporal: SQLite mantiene el archivo abierto. El cuerpo de la prueba pasó; falla la limpieza. En el CI, que corre en Linux, no ocurre. No lo persigas.

8. Invariantes que no se rompen

Si alguna vez ves lo contrario, no es un caso raro: es un fallo grave.

- **Toda consulta filtra por** `business_id`. Sin excepción.
- **Una factura emitida no se modifica.** Se corrige con una rectificativa.
- **Nada que mueva dinero se ejecuta sin confirmación** del titular.
- **La bitácora de seguridad es encadenada:** cada entrada firma la anterior.
- **Administración no abre el negocio de un cliente.** Gestiona acceso, no contenido.
- **El cerebro local sigue disponible** aunque toda IA externa falle.

Verificado sobre `main`: `web/server.py`, `web/deps.py`, `web/auth.py`, `web/scheduler.py`, `db.py`, `migrations.py`, `adapters/` y `pyproject.toml`.

Los números que cambian —esquema, recuento de pruebas— viven en `project-state.json`. Si este documento y el JSON no coinciden, manda el JSON.